Practical 9

Sr. No	Practical Question	Core Concept(s)	Type
1	Define a lambda function calc_cube = lambda x: x ** 3. Call it with the integer 4 and print the output.	lambda functions, Operators	**Easy**
2	Write a program using a for loop and an if-else condition to iterate from 1 to 15. Print "Odd" or "Even" for each integer.	Loops, Conditionals	**Easy**
3	Create a dictionary student = {"name": "Aman", "age": 20, "course": "CS"}. Use dictionary methods to print all keys and values safely.	Dictionaries	**Easy**
4	Import numpy as np. Create a 1D array using np.arange(start=10, stop=50, step=5). Print the array and its shape (.shape).	NumPy arange, Slicing parameters	**Easy**
5	Create a 3x3 matrix of float zeros using np.zeros() and a 2x4 matrix of integer ones using np.ones().	NumPy array creation (zeros, ones)	**Easy**
6	Given arr = np.array([5, 10, 15, 20, 25, 30, 35, 40]), extract every second element between index 1 and index 6 using start:stop:step slicing.	NumPy Slicing	**Easy**

7	Create a 3x3 2D NumPy array containing values from 1 to 9. Write expressions to access: (a) the complete 2nd row, (b) the complete 3rd column.	Matrix Row & Column Access	**Easy**
8	Given grid = np.array([[1, 2], [3, 4]]), modify the element at index [0, 1] to 20 and [1, 0] to 30. Print the updated matrix.	NumPy Array Modification	**Easy**
9	Given a = np.array([2, 4, 6]) and b = np.array([1, 3, 5]), perform element-wise addition (a + b) and element-wise multiplication (a * b).	NumPy Vectorized Arithmetic	**Easy**
10	Given a list of tuple pairs coords = [(3, 9), (1, 2), (5, 4)], use sorted() with a lambda key to sort them based on their second value.	Tuples, Lists, lambda	**Easy**
11	Given text = "PythonProgramming", use string slicing [start:stop:step] to extract every second character in reverse order.	String Slicing	**Moderate**
12	Create a 4x4 NumPy matrix containing numbers 1 to 16. Use slice modification grid[2, :] = 0 to set all elements in the 3rd row to 0.	NumPy Row Modification	**Moderate**
13	Write a user-defined function filter_dict_by_value(d, threshold) that returns a new dictionary containing only	Functions, Dictionaries, Loops	**Moderate**

	key-value pairs where the value exceeds threshold.		
14	Generate an array arr = np.arange(1, 21). Use boolean masking (arr[arr % 3 == 0]) to filter out and print all multiples of 3.	NumPy Boolean Masking	**Moderate**
15	Given a 2D array of shape (4, 4), use multi-dimensional slicing grid[1:3, 0:2] to extract a 2x2 sub-matrix from rows 1–2 and columns 0–1.	Sub-matrix Slicing	**Moderate**
16	Write a lambda function evaluate_score using a conditional expression that returns "Pass" if score >= 50 else "Fail". Test it with score = 65.	lambda, Control Flow	**Moderate**
17	Given dict1 = {"a": 10, "b": 20} and dict2 = {"b": 30, "c": 40}, write a function to merge them such that shared keys have their values added together.	Dictionaries, Custom Functions	**Moderate**
18	Create a 1D NumPy array of 10 zeros. Use slice assignment arr[1::2] = 9 to modify all odd-indexed elements to 9.	NumPy Step Assignment	**Moderate**
19	Write a program using np.ones((3, 3)) and add a 1D array np.array([10, 20, 30]) to it. Print the resulting matrix.	Array Addition	**Moderate**

20	Create a 3x4 2D NumPy array. Write code to extract the last row and reverse its elements using negative stepping [::-1].	2D Array Slicing & Reversing	**Moderate**

21. The Retail Store GST & Price Calculator

A retail store tracks item base prices in a dictionary:

inventory = {"Shirt": 800, "Jeans": 2200, "Jacket": 4500, "Cap": 300}.

- Extract the list of prices and convert it into a 1D NumPy array using np.array().
- Define a lambda function add_gst = lambda price: price * 1.18 to apply an 18% GST tax rate.
- Apply this lambda function across array elements using a list comprehension or loop to generate tax-inclusive prices.
- Update the inventory dictionary with the new calculated values and print it.

22. The Automated Classroom Attendance Grid

A digital attendance logger tracks student presence across 4 classes over 5 days using a 4x5 NumPy matrix initialized with np.zeros((4, 5), dtype=int) (where 0 = Absent, 1 = Present):

- Mark Class 1 (row 0) as fully present on all 5 days using row modification (matrix[0, :] = 1).
- Mark all classes as present on Day 3 (column 2) by setting matrix[:, 2] = 1.
- Calculate total attendance per class by performing array addition across columns (axis=1).
- Print the updated attendance matrix and the total attendance array per class.

23. The Dynamic Sales Commission Pipeline

An enterprise tracks quarterly performance for 3 sales representatives in a 3x4 2D NumPy array (3 reps across 4 quarters):

sales_matrix = np.array([[120, 150, 180, 200], [90, 110, 95, 130], [210, 240, 220, 250]]).

- Extract the sales data for Quarter 2 (column index 1) and Quarter 4 (column index 3) using column slicing.
- Define a lambda function calc_commission = lambda val: val * 0.05 and compute a 5% commission matrix via element-wise multiplication (sales_matrix * 0.05).
- Identify which representative exceeded 700 total annual units by performing row summation (np.sum(sales_matrix, axis=1)).

24. The Sound Signal Noise Threshold Filter

An audio sampling system records signal amplitude values in a 1D NumPy array generated using range boundaries:

signal = np.arange(start=-10, stop=10, step=2).

- Write a lambda function clip_negative = lambda val: 0 if val < 0 else val.
- Apply clipping directly using NumPy boolean modification (signal[signal < 0] = 0).
- Multiply the entire resulting array by a scalar gain factor of 1.5 using array multiplication.
- Print the initial, clipped, and amplified signal arrays.

25. The Smart Home Temperature Grid Monitor

An HVAC system logs temperature readings from 3 thermal sensors placed across 3 rooms using a 3x3 NumPy array:

temp_grid = np.array([[22.5, 24.0, 23.1], [26.8, 28.2, 27.5], [19.5, 21.0, 20.2]]).

- Extract middle room sensor readings (entire 2nd row: row index 1).
- Extract right wall sensor readings across all rooms (entire 3rd column: column index 2).
- Iterate through temp_grid.flatten() with a lambda filter expression is_overheated = lambda t: t > 25.0 to display warning flags for high temperatures.

26. The E-Commerce Product Catalog Sorter

An e-commerce app maintains inventory data in a list of tuples containing (Product_ID, Product_Name, Price):

catalog = [(101, "Laptop", 65000), (102, "Mouse", 750), (103, "Keyboard", 1500), (104, "Monitor", 18000)].

- Sort the catalog list in ascending order of Price using sorted() with a lambda key extracting index 2.
- Filter out items priced below 1000 using a for loop and store qualifying items in a dictionary filtered_catalog = {name: price}.
- Print the sorted list and the filtered dictionary.

27. The Image Processing Brightness Adjuster

A grayscale digital image tile of size 4x4 is represented by pixel intensity values stored in a 2D NumPy array:

pixels = np.array([[50, 60, 70, 80], [90, 100, 110, 120], [130, 140, 150, 160], [170, 180, 190, 200]]).

- Brighten the central 2x2 region (rows 1 to 2, columns 1 to 2) by adding scalar 50 to those specific pixels using slice modification.
- Create a 4x4 mask matrix of 1s using np.ones((4, 4)) scaled by 10, and perform array addition (pixels + mask).
- Print the modified pixel array.

28. The Multi-Branch Inventory Stock Synchronizer

Two distribution hubs track stock levels using dictionaries:

hub_A = {"Item_1": 100, "Item_2": 200, "Item_3": 150}

hub_B = {"Item_1": 50, "Item_2": 80, "Item_3": 120}.

- Convert the values of hub_A and hub_B into 1D NumPy arrays stock_A and stock_B.
- Perform element-wise addition (stock_A + stock_B) to compute total combined inventory.
- Define a lambda function check_reorder = lambda qty: "Reorder Required" if qty < 250 else "Sufficient" and apply it to each combined stock total in a for loop.

29. The Fleet GPS Tracker Coordinate Extractor

A logistics manager stores delivery truck positions as coordinates in a 5x2 2D NumPy array (5 trucks, [X, Y] coordinates):

coordinates = np.array([[12, 45], [28, 64], [35, 89], [42, 15], [50, 72]]).

- Extract all X-coordinates (1st column: index 0) using slicing coordinates[:, 0].
- Extract all Y-coordinates (2nd column: index 1) using slicing coordinates[:, 1].
- Select coordinate pairs for every alternate truck starting from the first truck using step slicing coordinates[::2, :].

30. The Financial Portfolio Return Matrix

A stock broker tracks daily percentage returns of 3 stocks over 4 days in a 3x4 2D NumPy array:

returns = np.array([[1.2, -0.5, 2.1, 0.8], [-1.0, -1.8, 0.4, 1.1], [0.5, 1.5, -0.2, 2.0]]).

- Use boolean masking (returns < 0) to find all negative return values and set them to 0.0.
- Convert all non-negative returns to basis points by multiplying the matrix by 100 via array multiplication.
- Calculate and print average returns per stock by computing row means using np.mean(returns, axis=1).